UniCaronas

Sistema de Caronas Universitárias

Relatório de Resultados — Sprint 2

Disciplina: Prática em Programação Web — 3ESCN

Equipe: Rafael Machado, Ariane Archanjo, Pedro Kafka, Matheus Sizanoski

---

1. Resumo Executivo

A Sprint 2 do projeto UniCaronas foi concluída com sucesso no período de 26 de março a 03 de abril de 2026. Todas as quatro histórias de usuário planejadas foram entregues dentro do prazo, totalizando 18 Story Points. O sistema passou a contar com o fluxo completo de caronas: criação, busca com filtros, solicitação de vaga e sugestão automática de valor por distância — tudo funcionando de ponta a ponta no navegador. Além disso, a equipe implementou um sistema de multilinguagem com suporte a Português, Inglês e Espanhol, entregando valor além do planejado.

Métrica	Valor
Story Points planejados	18 SP
Story Points entregues	18 SP (100%)
Histórias concluídas	4 de 4 (US04, US05, US06, US07)
Entrega adicional	Sistema de multilinguagem (PT / EN / ES)
Velocidade	9 SP / semana
Impedimentos resolvidos	1 de 1
Status geral	✅Sprint concluída

---

2. Histórias de Usuário Entregues

US04 — Criar carona (5 pontos)

Como motorista, quero publicar uma carona com trajeto, horário e vagas disponíveis para dividir os custos do trajeto com outros alunos.

Critério de Aceitação	Status
Campos origem, destino, horário, vagas e valor são obrigatórios	✅Implementado
Carona fica visível para outros usuários após publicação	✅Implementado
Valor sugerido calculado automaticamente ao informar a distância	✅Implementado
Motorista pode aceitar ou alterar o valor sugerido	✅Implementado

Critério de Aceitação	Status
Redireciona para detalhes da carona após publicação	✅Implementado

US05 — Sugestão de valor por distância (3 pontos)

Como motorista criando uma carona, quero receber uma sugestão de valor baseada no trajeto para ter uma referência justa ao definir o preço.

Critério de Aceitação	Status
Sistema calcula: valor = distancia_km × R$ 0,30	✅Implementado
Valor sugerido exibido automaticamente no formulário	✅Implementado
Usuário pode alterar o valor livremente	✅Implementado

US06 — Buscar caronas (5 pontos)

Como passageiro, quero buscar caronas disponíveis por origem e destino para encontrar uma compatível com meu trajeto.

Critério de Aceitação	Status
Filtros por origem, destino e data/horário funcionam corretamente	✅Implementado
Exibe motorista, horário, vagas disponíveis, valor e avaliação do motorista	✅Implementado
Caronas sem vagas ou canceladas não aparecem nos resultados	✅Implementado
Clicando em 'Ver detalhes' abre a página da carona	✅Implementado

US07 — Solicitar vaga em carona (5 pontos)

Como passageiro, quero solicitar uma vaga em uma carona disponível para garantir meu lugar no trajeto.

Critério de Aceitação	Status
Passageiro envia solicitação e vê confirmação na tela	✅Implementado
Motorista pode aceitar ou recusar via endpoint PATCH	✅Implementado
Vaga é reservada automaticamente ao aceitar (trigger no banco)	✅Implementado
Motorista não pode solicitar a própria carona	✅Implementado
Solicitação duplicada retorna erro 409	✅Implementado

---

3. Entrega Adicional — Sistema de Multilinguagem

Além das histórias planejadas, a equipe implementou suporte a múltiplos idiomas na interface frontend, entregando valor além do escopo original da Sprint.

O que foi implementado

Componente	Detalhes
Idiomas suportados	Português (padrão), Inglês, Espanhol
Seletor de idioma	Disponível na navbar em todas as telas autenticadas
Cobertura	Login, Cadastro, Dashboard, Buscar, Criar carona, Detalhes da carona
Persistência	Idioma salvo no localStorage — mantido entre sessões
Implementação	Módulo i18n.js com dicionário de traduções centralizado

Por que foi implementado

A equipe identificou que o projeto pode ser apresentado a uma audiência diversa — incluindo avaliadores e futuros usuários de outras instituições — e optou por antecipar essa funcionalidade, que estava prevista para backlog futuro. O esforço foi estimado em 3 pontos extras e a implementação não comprometeu as entregas obrigatórias da Sprint.

---

4. Telas Implementadas

Tela Criar Carona (criar-carona.html)

Formulário com campos de origem, destino, horário, vagas, distância e observações. Campo de distância dispara em tempo real o cálculo de sugestão de valor (R$ 0,30/km), exibindo o resultado em destaque com opção de aceitar ou editar. Botão de publicação com feedback visual de loading.

Tela Buscar Caronas (buscar.html)

Painel de filtros com origem, destino e data. Lista de resultados em cards com informações de rota, horário, distância, motorista com avaliação e preço por passageiro. Integração direta com o endpoint GET /api/caronas usando query strings para filtragem.

Tela Detalhes da Carona (carona.html)

Layout em duas colunas: coluna principal com detalhes da viagem e card do motorista (nome, avaliação, telefone); coluna lateral com preço em destaque e botão de solicitação de vaga. O botão é ocultado para o próprio motorista e exibido apenas enquanto a carona estiver ativa.

---

5. Principais Componentes de Código

5.1 Backend — Novos endpoints

Arquivo	Responsabilidade
controllers/caronasController.js	Lógica de criação, listagem, busca com filtros, solicitação e resposta de vaga
routes/caronas.js	Define endpoints: GET /, POST /, GET /:id, POST /:id/solicitar, PATCH /solicitacoes/:id

Arquivo	Responsabilidade
utils/precificacao.js	Funções calcularValorSugerido() e calcularTaxa() com configuração via variáveis de ambiente

5.2 Banco de Dados — Novas tabelas e triggers

Duas novas tabelas foram criadas: caronas (com status possíveis: ativa, em_andamento, concluida, cancelada) e solicitacoes_carona (com status: pendente, aceita, recusada, cancelada).

Dois triggers PostgreSQL foram implementados: trigger_vagas_carona atualiza vagas_disponiveis automaticamente ao aceitar ou cancelar solicitações; trigger_avaliacao_media recalcula a média do usuário a cada nova avaliação inserida.

5.3 Frontend — Módulo i18n.js

O módulo de internacionalização centraliza todos os textos da interface em um objeto de dicionário com três idiomas. A função t(key) retorna o texto no idioma ativo. A troca de idioma percorre todos os elementos com atributo data-i18n e substitui o conteúdo sem recarregar a página.

---

6. Impedimentos e Resoluções

Impedimento	Impacto	Resolução
Rota PATCH /solicitacoes/:id com conflito de path no Express	Endpoint retornava 404	Reordenação das rotas em caronas.js — rota estática /solicitacoes/:id declarada antes de /:id

---

7. Retrospectiva da Sprint

O que funcionou bem

- Fluxo completo criar carona → buscar → solicitar vaga funcionando end-to-end
- Triggers no banco eliminaram lógica de controle de vagas no backend
- Entrega adicional do sistema de multilinguagem dentro do prazo
- Definição clara dos critérios de aceitação facilitou validação das entregas

O que pode melhorar

- Iniciar testes manuais mais cedo no ciclo — deixar para os últimos dias aumenta risco
- Documentar endpoints no README à medida que são criados
- Adicionar validação de campos no backend (express-validator) — atualmente apenas no frontend

---

8. Tecnologias Utilizadas

Camada	Tecnologia	Versão / Detalhe
Frontend	HTML5 + CSS3 + JavaScript	Sem frameworks — Vanilla JS + i18n customizado
Backend	Node.js + Express	Express 4.x
Banco de Dados	PostgreSQL	v14+ — triggers automáticos
Autenticação	JWT (jsonwebtoken)	v9.0
Versionamento	GitHub	Branch principal com código estável